```
#!/usr/bin/python3
"""! Python Bridge for IFM IO-Link Master to Meassure over Modbus and send over Can.
    The information regarding usage of the script can be found in the readme.md in this folder.
    Information regarding installation and usage of Python 3 and the virtual environment can be found here:
    https://novo-wiki/xwiki/bin/view/Wissens-Db/Entwicklung/Tools/Python/

    Copycw Novotechnik Messwertaufnehmer OHG - All cws Reserved, June 2023
    Unauthorized copying of this file, via any medium is strictly prohibited.
    Proprietary and confidential.

 @section authors Author(s)
 - Created by %auteur% on %date%.
 @section Copyright
    Copyright Novotechnik Messwertaufnehmer OHG - All Rights Reserved, February 2023
    Unauthorized copying of this file, via any medium is strictly prohibited.
    Proprietary and confidential.

"""

'''
Notizen:
Bezug von Kanaele auf Register Ch1 = 1002 , Ch2 = 2002 usw . .
Diagnosedaten 1001 ,2001.
Diagnosedaten und Events 30 = ch1 ,40 =ch2, 100= ch8 | Seite 83
199 299 Statusinformationen
'''


__author__ = "Markus Steichele"
__credits__ = ["Markus Steichele"]
__version__ = "1.0.0"
__maintainer__ = "QE "
__email__ = "markus.steichele@novotechnik.de"
__status__ = "WIP"


import time
from datetime import datetime, timedelta
import time
from pyModbusTCP.client import ModbusClient
import xml.etree.ElementTree as ET
import struct
import can
from threading import Thread, Lock
from flask import Flask, render_template, request
from flask import Flask, send_file
from flask import Flask, make_response
from flask_socketio import SocketIO
from flask_caching import Cache
from waitress import serve
import subprocess
import psutil
from cantools.database import Database
from cantools.database.can import Message
from cantools.database.can.signal import Signal
from cantools.database.conversion import LinearConversion
import yaml



## @class Canbridge
#  @brief Hauptklasse zur Verwaltung einer CAN-Bridge zwischen IO-Link-Modulen und einem CAN-Netzwerk.
#  @details Diese Klasse übernimmt:
#  - Modbus-Kommunikation zu IO-Link-Modulen
```

```
69  #     - Auslesen von Prozess□, Diagnose□ und Eventregistern CRLF
70  #     - Identifikationsobjekte (acyclic read) CRLF
71  #     - Erstellung von DBC□Dateien CRLF
72  #     - CAN□Senden aller gesammelten Daten CRLF
73  class Canbridge:CRLF
74      CRLF
75      ## @brief Konstruktor der CAN□Bridge.CRLF
76      #  @details Initialisiert:CRLF
77      #     - CAN□Hardwareparameter CRLF
78      #     - Registerlisten CRLF
79      #     - Modbus□Timeouts CRLF
80      #     - Verbindungs□Cache CRLF
81      #     - DBC□Dateipfade CRLF
82      def __init__(self):CRLF
83          self.canhw = 'socketcan'CRLF
84          self.channel = 'vcan0'CRLF
85          self.bitrate = (250000) CRLF
86          self.id = 700CRLF
87          self.startid = 800CRLF
88          self.maxchn = 8CRLF
89          self.subnet = "192.168.1."CRLF
90          self.startip = 250CRLF
91          self.channelspermodul = 8 #Kanäle / ModulCRLF
92          self.module = 2#ModulanzahlCRLF
93          self.iddict = {}CRLF
94          self.maxid = NoneCRLF
95          self.iplist = []CRLF
96          self.messregister = [1002, 2002, 3002, 4002, 5002, 6002, 7002, 8002] #
            Messregister ohne PQI info. Obsolete!CRLF
97          self.diagregister = [1001, 2001, 3001, 4001, 5001, 6001, 7001, 8001] #
            Beinhaltet PQI & Messregister CRLF
98          self.eventregister = [30, 40, 50, 60, 70, 80, 90, 100] CRLF
99          self.busv = can.interface.Bus(bustype='socketcan', channel='vcan0',
            bitrate=self.bitrate) #can.interface.Bus('virtual',bustype='virtual') CRLF
100         self.masterIP = "192.168.1.250"CRLF
101         self.port = 502CRLF
102         self.iodevice = None#ModbusClient(host=self.masterIP, port=self.port,
            auto_open=True, timeout = 0.02)CRLF
103         self.running = FalseCRLF
104         self.cycle = 1CRLF
105         self.dlfileTemp = "static/Canbride.dbc" CRLF
106         # self.dlfileTemp = "/home/pi/mycanlogger/config/config.dbc" # test workflow
            handling CRLF
107         self.dlfile = "uploads/Canbride.dbc"CRLF
108         self.modbus_lock = Lock() # VorbereitungCRLF
109         self.pause_modbus = 0.01CRLF
110         self.lastip = self.masterIPCRLF
111         self.connection_cache = {}CRLF
112         self.status = NoneCRLF
113         self.counter = 10000CRLF
114         self.deactivadedID = []CRLF
115 CRLF
116     CRLF
117     ## @brief Konvertiert Integerwerte in Bytes.CRLF
118     #  @param values Liste von Integerwerten.CRLF
119     #  @return Liste aus Bytewerten.CRLF
120     #  @throws ValueError falls Werte außerhalb des gültigen Bereichs liegen.CRLF
121     def int_to_bytes(self, values):CRLF
122         if len(values) != 4:CRLF
123             raise ValueError("Die Liste muss genau zwei Werte enthalten.")CRLF
124         CRLF
125         for value in values:CRLF
126             if not (0 <= value < 2**32):CRLF
127                 raise ValueError("Jeder Wert muss zwischen 0 und 2^32-1 liegen.")CRLF
128         CRLF
129         byte_list = []CRLF
130         for value in values:CRLF
131             byte_data = struct.pack('<H', value)CRLF
132             byte_list.extend(byte_data)CRLF
133         return byte_listCRLF
134 CRLF
135     ## @brief Wählt je nach Register die relevanten Werte aus.CRLF
136     #  @param valuelist Eingelesene Modbus□Registerdaten.CRLF
```

```
    #  @param register Modbus-Registeradresse.
    #  @return Gefilterte Liste.
    def register_range(self,valuelist, register):
        if register >= 101:
            valuelist = valuelist[0:4]
        else:
            valuelist = [valuelist[0], valuelist[1], valuelist[3], valuelist[5]]
        return(valuelist)

    ## @brief Holt oder erzeugt eine cached Modbus-TCP-Verbindung.
    #  @param ip Modul-IP.
    #  @return ModbusClient-Instanz.
    def get_connection(self, ip):
        if ip not in self.connection_cache:
            self.connection_cache[ip] = ModbusClient(host=ip, port=self.port,
            auto_open=True, timeout= self.cycle * 1.2)
        return self.connection_cache[ip]

    ## @brief Liest Prozess- oder Diagnosedaten eines IO-Link-Ports aus.
    #  @details Beinhaltet PQI-Prüfung, rekursives Auslesen und Fehlerbehandlung.
    #  @param id Identifikationsnummer / CAN-ID.
    #  @param retry Retry-Zähler.
    def iodata(self, id, retry=0):
        ''' Ließt das Prozessdatenregister bis PQI Byte einen Fehler
        meldet. Dann wird self.detect_thread_io_data() Aufgerufen um die Events zu
        lesen
        '''
        valuelist = None
        ip = self.iddict[id][0]
        register = self.iddict[id][1]
        iodevice = self.get_connection(ip)
        try:
            if not register in self.eventregister or retry is False :
                with self.modbus_lock: # Vorbereitung
                    valuelist = iodevice.read_holding_registers(register, 6)
                    if valuelist is not None:
                        if not any(getattr(item, "isError", lambda: False)() for item
                        in valuelist):
                            valuelist = self.register_range(valuelist, register)
                            self.iddict[id][2] = self.int_to_bytes(valuelist)
                            if valuelist[0] >= 2: # PQI Check
                                evthread = Thread(target=self.iodata, args=(id+8,
                                False,))
                                evthread.start()
                        else:
                            print("valuelist is Error")
                            valuelist = [255, 255, 255, 255, 255, 255, 255, 255]
                            self.iddict[id][2] = valuelist
                    else:
                        failid = self.startid - 1
                        # ID sicher auf 16 Bit begrenzen
                        elog_id = id & 0xFFFF
                        elog = self.int_to_bytes([elog_id, 0, 0, 0])
                        self.iddict[failid][2] = elog

        except Exception as e:
            print(e)

        

    ## @brief Führt paralleles Auslesen aller IO-Daten durch.
    #  @param detect Wenn True, alternativer Thread-Startmodus.
    def thread_io_data(self, detect = False):
        for id in self.iddict.keys():
            if not id in self.deactivadedID:
                if detect == False:
                    rv = Thread(target=self.iodata, args=(id,))
                else:
                    # wird nur einmalig bei der initialisierung ausgeführt um den
                    Portstatus ( belegt, nicht belegt, health ) zu bekommen
                    rv = Thread(target=self.iodata, args=(id, False,))
                rv.start()
                time.sleep(self.pause_modbus)
    
```

```
205     ## @brief Erstellt IP-Liste aus Subnetz, Startadresse & Modulanzahl.
206     def setip(self):
207         for ip in range(0, self.module):
208             modulip = self.startip + ip
209             self.iplist.append(self.subnet + str(modulip))
210 
211             
212     ## @brief Baut vollständiges ID-Mapping (ID → IP → Register → Bytewerte).
213     def copy_temp_to_upload(self):
214         with open(self.dlfileTemp, "rb") as src:
215             with open(self.dlfile, "wb") as dst:
216                 dst.write(src.read())
217 
218     ## @brief Berechnet maximale Kanal- und ID-Anzahl.
219     def setmaxchannels(self):
220         self.maxchn = self.channelspermodul * self.module
221         self.maxid = self.maxchn * 2
222 
223     ## @brief Baut vollständiges ID-Mapping (ID → IP → Register → Bytewerte).
224     #  @details Ruft im Anschluss dict_to_dbc() auf.
225     def set_id_dict(self):
226         dummybytelist = [255, 255, 255, 255, 255, 255, 255, 255]
227         kanalregister = self.diagregister + self.eventregister
228         for modul in range(self.module):
229             register = 0
230             ip = self.iplist[modul]
231             self.status = f"Modul mir der Adresse {ip} wird konfiguriert"
232             print(f"Modul: {modul}, IP: {ip}")  # Debugging-Ausgabe
233             if self.module == 1:
234                 lastid = float(self.maxid / self.module)
235             lastid = float(self.maxid / self.module)
236             for id in range(int(lastid)):
237                 self.iddict[self.startid + id + modul * int(self.maxid /
                    self.module)] = [ip, kanalregister[register], dummybytelist]
238                 print(f"{id} {self.startid + id + modul * int(self.maxid /
                    self.module)} { kanalregister[register]}")
239                 register = (register + 1) % 24
240             modul = modul + 1
241         self.dict_to_dbc()
242         
243     ## @brief Initialisiert CAN-Hardwareobjekt.
244     def inithw(self):
245         self.bus = can.interface.Bus(bustype=self.canhw, channel=self.channel,
            bitrate=250000)
246         
247     ## @brief Sendet alle gesammelten Messdaten als CAN-Nachrichten.
248     def send_dict_to_can(self):
249         for key , value in self.iddict.items():
250             if not key in self.deactivadedID:
251                 if not value[2] == None:
252                     msg = can.Message(arbitration_id=key, data=value[2])
253                     self.bus.send(msg)
254                     # time.sleep(self.pause_modbus)
255 
256     ## @brief Schreibt Identifikationsdaten eines Sensors als XML.
257     #  @param data Dictionary der IO-Link-Identifikationsobjekte.
258     def writexml(self, data):
259         aliasDict = {16: "Vendor_Name", 17: "Vendor_Text", 18: "Product_Name", 19:
            "Product_ID", 20: "Product_Text", 21: "Serial_Number"}
260         if data[21] is not None or not data[21] == "":
261             node_id = data[21]
262             filename = f"SN{node_id}.xml"
263             root = ET.Element("root")
264             for key, value in data.items():
265                 element = ET.SubElement(root, aliasDict[key])
266                 element.text = str(value)
267                 print(str(key) + "||" +str(value))
268             tree = ET.ElementTree(root)
269             if not filename == "SN.xml":
270                 tree.write(f"./uploads/{filename}", encoding='utf-8',
                    xml_declaration=True )
271             print(filename)
272     
```

```
    ## @brief Führt IO-Link Acyclic Read (Identifikationsobjekte) aus.
    # @param hostip IP des IO-Link-Masters.
    # @param port IO-Link-Port.
    # @return Dictionary der gelesenen Identifikationsobjekte.
    def iolink_identifikation(self, hostip, port):
        identDict = {}
        client = self.get_connection(hostip)
        index_list = [16, 17, 18, 19, 20, 21]
        subindex = 0
        for index in index_list:
            print(f"\n--- Abfrage Index {index} ---")
            request = [0] * 22
            request[0] = port
            request[1] = index
            request[2] = subindex
            request[3] = (1 << 8) | 1
            request[4] = 0

            client.write_multiple_registers(500, request)
            time.sleep(0.2)

            response = client.read_holding_registers(0, 22)

            if response:
                hex_string = ''.join(f'{r:04x}' for r in response[6:22])
                try:
                    ascii_string = bytes.fromhex(hex_string).decode('ascii',
                    errors='ignore').strip('\x00')
                    identDict[index] = ascii_string.replace("/","_").replace("\x00",
                    "").replace("OH","").replace("\x01", "").replace(" ",
                    "").replace(".", "").replace(":", "").replace("-",
                    "_").replace("\t", ", ")
                    print("ASCII:", ascii_string)
                except Exception as e:
                    print("Fehler beim Dekodieren:", e)
            else:
                print("Keine Antwort erhalten.")
            client.write_multiple_registers(500, [0]*5)
            time.sleep(0.10)
        client.close()
        self.writexml(identDict)
        return identDict

    ## @brief Bereinigt Rohsensornamen für DBC-Erstellung.
    # @param sensorraw Ursprungsstring.
    # @return Bereinigter Name.
    def clean_sensor_name(self, sensorraw):
        sensor = sensorraw.replace("/","_").replace("OH","").replace(" ",
        "").replace(".", "").replace(":", "").replace("-", "_").replace("\t", ",
        ")
        return sensor

    ## @brief Lädt YAML-Konfigurationen für Sensoren.
    # @param path Pfad zur YAML-Datei.
    # @return Parsed YAML Struktur.
    def load_sensor_config(self, path="knownSensors.yaml"):
        with open(path, "r") as f:
            return yaml.safe_load(f)

    ## @brief Erstellt ein DBC-Signalobjekt.
    # @param cfg YAML-Konfiguration.
    # @param startbit Bitoffset.
    # @param signalname Signalname.
    # @param comment Kommentartext.
    # @return Cantools Signalobjekt.
    def build_signal(self, cfg, startbit, signalname, comment):
        pfix=None
        pfix = cfg.get("pfix")
        signalname=cfg.get("name", signalname)
        if not pfix == None:
            signalname = f"{pfix}{signalname[2:]}"
        signalname = signalname[:32]
        signal = Signal(
```

```
            name=signalname,CRLF
            start=startbit,CRLF
            length=16,CRLF
            byte_order="little_endian",CRLF
            is_signed=cfg["is_signed"],CRLF
            conversion=LinearConversion(CRLF
                scale=cfg["scale"],CRLF
                offset=cfg["offset"],CRLF
                is_float=FalseCRLF
            ),CRLF
            minimum=-3000,CRLF
            maximum=3000,CRLF
            unit=cfg["unit"],CRLF
            comment=commentCRLF
        )CRLF
        return signalCRLF
CRLF
    ## @brief Ermittelt passende Sensorkonfiguration oder Default□Signal.CRLF
    # @param signalname Ursprungs□Signalname.CRLF
    # @param value_startbit Bitoffset.CRLF
    # @param signalcomment Kommentartext.CRLF
    # @param iddict Identifikationsdaten.CRLF
    # @param register IO□Link□ oder Event□Register.CRLF
    # @return DBC□Signalobjekt.CRLF
    def check_known_sensors(self, signalname, value_startbit, signalcomment, iddict,
    register):CRLF
        sensor_base = self.clean_sensor_name(iddict[19])CRLF
        if register in self.eventregister:CRLF
            sensor_base = "diag"CRLF
            startbit = value_startbit + 64CRLF
        else:CRLF
            startbit = value_startbitCRLF
        sensor_key = f"{sensor_base}_{startbit}"CRLF
        config = self.load_sensor_config()CRLF
        cfg = config.get(sensor_key, config["defaultSignal"])CRLF
        return self.build_signal(CRLF
            cfg=cfg,CRLF
            startbit=value_startbit,CRLF
            signalname=signalname,CRLF
            comment=signalcommentCRLF
        )CRLF
CRLF
    ## @brief Wandelt IO□Daten vollständig in eine DBC□Datei um.CRLF
    # @details Für jede ID:CRLF
    #  - Identifikationsdaten lesen CRLF
    #  - Signale erzeugen CRLF
    #  - Nachrichtenlänge erzeugen CRLF
    #  - In Datenbank aufnehmen CRLF
    #  - DBC schreiben CRLF
    def dict_to_dbc(self):CRLF
        portDict = {}CRLF
        db = Database()CRLF
        def signal_add(id, idict, register): CRLF
            signallist = []CRLF
            start = {0:0, 1:16, 2:32, 3:48}CRLF
            for key, value in start.items():CRLF
                signalname = f'SignalID_{id}_{key}'CRLF
                signalcomment = ""CRLF
                try:CRLF
                    tempname = f"SN{idict[21]}ID{id}S{key}"CRLF
                    if not tempname.startswith("SNID"):CRLF
                        signalname = tempnameCRLF
                        signalname = signalname.replace("/","_").replace("\x00",
                        "").replace("OH","").replace("\x01", "").replace(" ",
                        "").replace(".", "").replace(":", "").replace("-",
                        "_").replace("\t", ", ")CRLF
                        for cvalue in idict.values():CRLF
                            comment = cvalue.replace("/","_").replace("\x00",
                            "").replace("OH","").replace("\x01", "").replace(" ",
                            "").replace("\t", ", ")CRLF
                            if not comment == "" or not comment == None:CRLF
                                signalcomment = f"SN{idict[21]}"CRLF
                            else:CRLF
```

```
                                        signalcomment = f"IOLink {str(signalname)}"
                    except Exception as e:
                        print(e)
                    signal = self.check_known_sensors(signalname, value, signalcomment,
                    idict, register)
                    signallist.append(signal)
                    self.status = f"{signalname} zur dbc hinzugefügt"
                return signallist

        for key, value in self.iddict.items():
            identDict = None
            if value[1] in self.diagregister:
                port = int((value[1]-1)/1000)
                identDict = self.iolink_identifikation(value[0], port)
                portDict[port] = identDict
            if value[1] in self.eventregister:
                port = int((value[1]/10)-2)
                identDict = portDict[port]
            signallist = signal_add(key, identDict, value[1])
            max_bit_position = max(signal.start + signal.length for signal in
            signallist)
            message_length = (max_bit_position + 7) // 8
            message = Message(
                frame_id=key,
                name=f'ExampleMessage{key}',
                length=message_length,
                signals=signallist
            )
            try:
                db.messages.append(message)
            except Exception as e:
                print(e)
                pass

        with open(self.dlfile, 'w', encoding="iso-8859-1") as f:
            self.status = f"{f} wird ins DBC übernommen"
            f.write(db.as_dbc_string())


    ## @brief Hauptschleife: IO lesen → CAN senden → Zykluszeit warten.
    def runbridge(self):
        while self.running == True:
            z1 = datetime.now()
            t1 = Thread(target=self.thread_io_data)
            t2 = Thread(target=self.send_dict_to_can)
            t1.start()
            offset = (datetime.now() - z1).total_seconds()
            sleep_time = max(0, self.cycle - offset - 0.004)
            time.sleep(sleep_time)
            t1.join() #Warten um polling issus zu eleminieren
            t2.start()


## @class CanbridgeWeb
#  @brief Webinterface zur Steuerung der Canbridge.
#  @details
#    Stellt ein Flask‑Webinterface bereit, verwaltet Routing, Konfiguration,
#    Systeminformationen, Modul‑Setup, CAN‑Konfiguration und Download von
DBC‑Dateien.
class CanbridgeWeb:
    ## @brief Konstruktor der Weboberfläche.
    #  @details Initialisiert:
    #    - Flask‑App
    #    - Canbridge‑Instanz
    #    - Routen
    #    - Systeminformationen (Hostname, IP, OS)
    #    - Dropdown‑Parameter (Baudrate, Module, Start‑IP, etc.)
    #    - interne Statusvariablen
    def __init__(self):
        self.app = Flask(__name__)
        self.cb = Canbridge()
        self.cs = None
```

```
        self.add_routes()CRLF
        self.cg = NoneCRLF
        self.comments = {}CRLF
        self.candb = NoneCRLF
        self.myip = NoneCRLF
        self.status = self.cb.statusCRLF
        self.osname = self.get_osname()CRLF
        self.myip = self.get_myip()CRLF
        self.hostname = self.get_myhostname()CRLF
        self.modulcount = {1:1, 2:2, 3:3, 4:4, 5:5, 6:6, 7:7, 8:8}CRLF
        self.subnet = "192.168.1."CRLF
        self.canids = {800:800, 700:700, 600:600, 500:500, 400:400, 300:300, 200:200,
        100:100}CRLF
        self.canhw = {"Socketcan" : "socketcan" } CRLF
        self.knoten = {"Can0":"can0" , "VitualCan" : "vcan0"}CRLF
        self.bitrate = {"50kBaud" : 50000, "125kBaud" : 125000, "250kBaud" : 250000 ,
        "500kBaud" : 500000, "800kBaud" : 800000, "1000kBaud" : 1000000} CRLF
        self.freeRam = self.get_free_mem()CRLF
        self.diskusage = self.get_free_space()CRLF
        self.cycle = {"1000ms" : 1.0, "500ms" : 0.5, "250ms" : 0.25, "100ms" : 0.1,
        "50ms" : 0.05, "30ms" : 0.03, "10ms" : 0.01}CRLF
        self.function = {"config" : 0, "show" : 1, "run" : 2, "Stop" : 3}CRLF
        self.startip = {245:245, 246:246, 247:247, 248:248, 249:249, 250:250,
        251:251, 252:252, 253:253, 254:254, 255:255}CRLF
        self.cb.id = 700CRLF
        self.cb.startid = 800CRLF
        self.cb.maxchn = 8CRLF
        self.cb.subnet = "192.168.1."CRLF
        self.cb.startip = NoneCRLF
        self.channelspermodul = 8CRLF
        self.cb.module = 2CRLF
        self.dlfile = self.cb.dlfileCRLF
        self.savefile = NoneCRLF
        self.belegung = NoneCRLF
        self.cycletime = NoneCRLF
CRLF
CRLF
    ## @brief Ermittelt die lokale IPv4□Adresse aus dem System.CRLF
    #  @return IP□Adresse oder None bei Fehler.CRLF
    def get_myip(self):CRLF
        time.sleep(1)CRLF
        try:CRLF
            command = "ip addr show eth0 | grep 'inet ' | awk '{print $2}' | cut -d/
            -f1 | head -n 1"CRLF
            result = subprocess.run(command, shell=True, capture_output=True,
            text=True, check=True)CRLF
            return result.stdout.strip()CRLF
        except subprocess.CalledProcessError as e:CRLF
            print(f"Error executing command: {e}")CRLF
            self.getstatus(e)CRLF
            self.get_myip()CRLF
            return NoneCRLF
CRLF
    ## @brief Liefert den Hostnamen des Systems.CRLF
    #  @return Hostname oder None bei Fehler.CRLF
    def get_myhostname(self):CRLF
        try:CRLF
            command = "hostname"CRLF
            result = subprocess.run(command, shell=True, capture_output=True,
            text=True, check=True)CRLF
            return result.stdout.strip()CRLF
        except subprocess.CalledProcessError as e:CRLF
            print(f"Error executing command: {e}")CRLF
            self.getstatus(e)CRLF
            return NoneCRLF
       CRLF
    ## @brief Liefert den Betriebssystemnamen.CRLF
    #  @return "Linux", "Windows" oder OS□String.CRLF
    def get_osname(self):CRLF
        try:CRLF
            command = "uname"CRLF
            result = subprocess.run(command, shell=True, capture_output=True,
            text=True, check=True)CRLF
```

```
            return result.stdout.strip()CRLF
        except subprocess.CalledProcessError as e:CRLF
            print(f"Error executing command: {e}")CRLF
            self.getstatus(e)CRLF
            return("Windows")CRLF
    CRLF
    ## @brief Auswahl des passenden CAN-Treibers je nach OS.CRLF
    def get_driver(self):CRLF
        if self.osname == "Linux":CRLF
            self.canhw = {"Socketcan" : "socketcan" } CRLF
            self.knoten = {"Can0":"can0", "VCan0":"vcan0" }CRLF
        else:CRLF
            self.canhw = {"Ixxat" : "ixxat" , "Peak" : 'pcan'} CRLF
            self.knoten = {"Channel 0" : "0" }CRLF
CRLF
    ## @brief Priorisiert Modbus-TCP Port 502 mittels Shellskript.CRLF
    def prio_modbus(self):CRLF
        time.sleep(0.1)CRLF
        try:CRLF
            command = "bash qosPrio502.sh "CRLF
            result = subprocess.run(command, shell=True, capture_output=True,
            text=True, check=True)CRLF
            print(result.stdout.strip())CRLF
        except subprocess.CalledProcessError as e:CRLF
            print(f"Error executing command: {e}")CRLF
            self.getstatus(f"Fehler {e} beim Priorisieren des Port 502")CRLF
            return NoneCRLF
      CRLF
    ## @brief Dummy-Endlosschleife als Thread-Hintergrundprozess.CRLF
    def start_loop(self):CRLF
        '''CRLF
        ThreadsCRLF
        '''CRLF
        i = 0CRLF
        while True:CRLF
            time.sleep(1)CRLF
    CRLF
    ## @brief Liefert freien Speicherplatz auf dem Root-Filesystem.CRLF
    #  @return Prozentwerte der Belegung.CRLF
    def get_free_space(self):CRLF
        disk_usage = psutil.disk_usage('/')CRLF
        return(disk_usage.percent)CRLF
CRLF
    ## @brief Liefert freien RAM in MB.CRLF
    #  @return Freier RAM in Megabyte.CRLF
    def get_free_mem(self):CRLF
        mem = psutil.virtual_memory()CRLF
        free_ram = mem.availableCRLF
        free_ram_mb = round(free_ram / (1024 * 1024))CRLF
        return(free_ram_mb)CRLF
CRLF
    ## @brief Registriert HTTP-Routen der Flask-App.CRLF
    def add_routes(self):CRLF
        ''' legt das Routing der url festCRLF
        '''CRLF
        self.app.add_url_rule('/', view_func=self.index, methods=['GET', 'POST'])CRLF
        self.app.add_url_rule('/download', view_func=self.downloadfile,
        methods=['GET'])CRLF
CRLF
    ## @brief Index-Seite, nimmt Web-Kommandos entgegen.CRLF
    def index(self):CRLF
        ''' reciving the commands from webinterface'''CRLF
        if 'transmit' in request.form:CRLF
            self.read_post_setup()CRLF
        return render_template('index.html', cs=self)CRLF
     CRLF
    ## @brief Konfiguriert Linux-CAN-Interface über Bash.CRLF
    #  @param interface Name des Interfaces.CRLF
    #  @param bitrate CAN-Baudrate.CRLF
    def bash_set_can_interface(self, interface, bitrate = 500000):CRLF
        ''' configure the caninterface in Linux '''CRLF
        if interface == "vcan0":CRLF
            try:CRLF
```

```
613                 subprocess.run(f"sudo ip link set {interface} down", shell=True,
                    check=True, capture_output=True, text=True)CRLF
614                 try:CRLF
615                     subprocess.run("sudo ip link add dev vcan0 type vcan",
                        shell=True, check=True, capture_output=True, text=True)CRLF
616                 except:CRLF
617                     passCRLF
618                 subprocess.run("sudo ip link set up vcan0" , shell=True, check=True,
                    capture_output=True, text=True)CRLF
619                 result = subprocess.run(f"ip -details link show {interface}",
                    shell=True, check=True, capture_output=True, text=True)CRLF
620                 print(result.stdout)CRLF
621             except subprocess.CalledProcessError as e:CRLF
622                 print(f"An error occurred while setting the CAN interface:
                    {e.stderr}")CRLF
623                 raiseCRLF
624         else:CRLF
625             try:CRLF
626                 subprocess.run(f"sudo ip link set {interface} down", shell=True,
                    check=True, capture_output=True, text=True)CRLF
627                 subprocess.run(f"sudo ip link set {interface} up ", shell=True,
                    check=True, capture_output=True, text=True)CRLF
628                 result = subprocess.run(f"ip -details link show {interface}",
                    shell=True, check=True, capture_output=True, text=True)CRLF
629                 print(result.stdout)CRLF
630             except subprocess.CalledProcessError as e:CRLF
631                 print(f"An error occurred while setting the CAN interface:
                    {e.stderr}")CRLF
632                 raiseCRLF
633 CRLF
634     ## @brief Liest POST□Parameter des Webinterfaces und wandelt sie in ein dict
        um.CRLF
635     def read_post_setup(self):CRLF
636         ''' recives commands'''CRLF
637         cmddict = {}CRLF
638         options = ['function', 'modulcount', 'startip', 'canids', 'cycle',
            'knoten']CRLF
639         for option in options:CRLF
640             try:CRLF
641                 if option == 'knoten':CRLF
642                     cmddict[option] = request.form[option]CRLF
643                 else:CRLF
644                     if option == 'cycle':CRLF
645                         cmddict[option] = float(request.form[option])CRLF
646                     else:CRLF
647                         cmddict[option] = int(request.form[option])CRLF
648             except Exception as e:CRLF
649                 print(e)CRLF
650                 self.getstatus(e)CRLF
651         print(cmddict)CRLF
652         self.ex(cmddict)CRLF
653 CRLF
654     ## @brief Konfiguriert die CAN□Bridge anhand der Web□Parameter.CRLF
655     #  @param cmddict Dictionary mit Setup□Parametern.CRLF
656     def config(self, cmddict):CRLF
657         self.cb = Canbridge()CRLF
658         self.cb.running = FalseCRLF
659         status = NoneCRLF
660         try:CRLF
661             self.cb.iddict = {}CRLF
662             self.cb.startip = cmddict['startip']CRLF
663             self.cb.module = cmddict['modulcount']CRLF
664             self.cb.startid = cmddict['canids']CRLF
665             self.cb.cycle = cmddict['cycle']CRLF
666             self.cb.channel = cmddict['knoten']CRLF
667             self.cb.setip()CRLF
668             self.cb.setmaxchannels()CRLF
669             self.cb.inithw()CRLF
670             self.bash_set_can_interface(cmddict['knoten'])CRLF
671             self.cb.copy_temp_to_upload()CRLF
672             config = Thread(target=self.cb.set_id_dict , args=())CRLF
673             config.start()CRLF
674             while config.is_alive():CRLF
```

```
675                     self.getstatus(f"{self.cb.status}")
676                 self.detect()
677                 self.setdeactivaded()
678                 view =
                    self.check_min_cycleTime()
679                 self.show(view)
680
681             except Exception as e:
682                 print(e)
683                 self.getstatus(e)
684
685         ## @brief Berechnet minimale Kanal-Cycle-Time basierend auf aktiven Ports.
686         #  @return Zeit pro Port.
687         def check_min_cycleTime(self):
688             posiblePorts = len(self.cb.iddict.items()) * 0.5
689             deactivadedPorts = len(self.cb.deactivadedID) * 0.5
690             activePorts = (posiblePorts - deactivadedPorts)
691             cyclePerPort = self.cb.cycle / activePorts
692             self.cb.pause_modbus = (self.cb.cycle * 0.5) / activePorts
693             self.cycletime = cyclePerPort
694             view = (
695             f"iddict={len(self.cb.iddict)}, "
696             f"deact={len(self.cb.deactivadedID)}, "
697             f"possible={posiblePorts}, "
698             f"active={activePorts}, "
699             f"cycle={self.cb.cycle}, "
700             f"cycletime/Port={cyclePerPort}, "
701             f"pause_modbus={self.cb.pause_modbus}"
702             )
703             return view
704
705         ## @brief Führt eine Sensor-Belegungserkennung durch.
706         #  @return dict mit (IP, Kanal) → Status
707         def detect(self):
708             belegung = {}
709             kanal = 1
710             self.cb.thread_io_data(detect=True)
711             for key, value in self.cb.iddict.items():
712                 if value[1] in self.cb.diagregister:
713                     belegung[(value[0], kanal)] = value[2][0]
714                 kanal = kanal + 1
715                 if kanal > 8:
716                     kanal = 1
717             self.belegung = belegung
718
719         ## @brief deaktiviert nicht belegte Kanäle.
720         def setdeactivaded(self):
721             kanal = 1
722             try:
723                 for key, value in self.cb.iddict.items():
724                     port = self.belegung[(value[0], kanal)]
725                     if port == 7:
726                         self.cb.deactivadedID.append(key)
727                         self.getstatus(f"{key} wurde deaktiviert")
728                     kanal = kanal + 1
729                     if kanal > 8:
730                         kanal = 1
731             except Exception as e:
732                 self.getstatus(e)
733
734         ## @brief Zeigt die CAN-zu-IO-Link-Zuordnung an.
735         def show(self, head = ""):
736             ''' show the canbridge '''
737             status = {1:"I.O.", 5:"SenErr", 7:"n.c.", 255:"IPErr"}
738             belegung = self.belegung.copy()
739             kanal = 1
740             view = f"{head}\nMapping Canmatrix zu I.O. Linkmaster\n"
741             view = f"{view}\tCanId \t : \t Port \t : \t IP Modul \t : \t State \t : \t
                Reg \t\n"
742             try:
```

```
743             for key, value in self.cb.iddict.items():CRLF
744                 port = belegung[(value[0], kanal)]CRLF
745                 if not port == 255:CRLF
746                     view = (f"{view}\t {key} \t : \t {kanal} \t : \t {value[0]} \t :
                        \t {status[port]} \t : \t {value[1]} \t\n")CRLF
747                 else:CRLF
748                     view = (f"{view}\t {key} \t : \t {kanal} \t : \t {value[0]} \t :
                        \t {status[port]} \t : \t {value[1]} \t\n")CRLF
749                 kanal = kanal + 1CRLF
750                 if kanal > 8:CRLF
751                     kanal = 1CRLF
752             if self.cycletime < 0.01:CRLF
753                 error = (CRLF
754                     f'<font color="#FFC72C">'CRLF
755                     f'Achtung: Abtastrate {self.cb.cycle * 1000:.0f} ms → 'CRLF
756                     f'max. {int(self.cb.cycle / 0.01)} Kanäle.<br>'CRLF
757                     f'Aktuell: {self.cycletime * 1000:.0f} ms/Kanal – mögliche
                        Verluste.<br>'CRLF
758                     f'Messung kann unter Berücksichtigung dieser Meldung fortgesetzt
                        werden.<br>'CRLF
759                     f'</font>'CRLF
760                 )CRLF
761                 view = (f"{error}{view}")CRLF
762             self.getstatus(view)CRLF
763         except:CRLF
764             self.getstatus("Error, bitte prüfen ob Sensor angeschlossen")CRLF
765 CRLF
766     ## @brief Startet die CAN□Bridge (Loop).CRLF
767     def run_bridge(self):CRLF
768         ''' start the canbridge '''CRLF
769         self.cb.running = TrueCRLF
770         r = Thread(target=self.cb.runbridge, daemon=False)CRLF
771         r.start()CRLF
772         self.getstatus("Bridge aktiv")CRLF
773     CRLF
774     ## @brief Stoppt die CAN□Bridge.CRLF
775     def stop_bridge(self):CRLF
776         ''' stop the canbridge '''CRLF
777         self.cb.running = FalseCRLF
778         self.getstatus("Bridge gestopt")CRLF
779 CRLF
780     ## @brief Führt konfigurierte Funktion abhängig vom Web□Command aus.CRLF
781     #  @param cmddict Parameterdict.CRLF
782     def ex(self,cmddict):CRLF
783         if cmddict['function'] == 0:CRLF
784             self.savefile = TrueCRLF
785             self.stop_bridge()CRLF
786             self.cb.__init__()CRLF
787             self.config(cmddict)CRLF
788 CRLF
789         if cmddict['function'] == 1:CRLF
790             self.savefile = TrueCRLF
791             self.show()CRLF
792 CRLF
793         if cmddict['function'] == 2:CRLF
794             self.prio_modbus()CRLF
795             self.run_bridge()CRLF
796 CRLF
797         if cmddict['function'] == 3:CRLF
798             self.savefile = NoneCRLF
799             self.stop_bridge()CRLF
800 CRLF
801     ## @brief Stellt DBC□Datei via Webdownload bereit.CRLF
802     def downloadfile(self):CRLF
803         ''' filedownload selection '''CRLF
804         if self.dlfile[-4:] == '.dbc':CRLF
805             response = make_response(send_file(self.dlfile, as_attachment=True,
                download_name=self.dlfile))CRLF
806             response.headers["Content-Type"] = "application/xml"CRLF
807             response.headers["Content-Disposition"] = "attachment; filename=" +
                str(self.dlfile)CRLF
808             self.getstatus(str(self.dlfile) + " wurde runter geladen")CRLF
809         return responseCRLF
```

```
CRLF
    ## @brief Setzt neuen Status zur Anzeige im Webinterface. CRLF
    # @param status Text des neuen Status. CRLF
    def getstatus(self, status): CRLF
        self.status = status CRLF
        CRLF
## @brief Startet die Flask Webapplikation. CRLF
    # @details CRLF
    #     - Startet Webserver auf Port 8082 CRLF
    #     - Debugmodus jederzeit aktivierbar CRLF
    #     - Optionaler Loop Thread deaktiviert CRLF
    def run(self): CRLF
        ''' Startet die Webapp CRLF
            Aukommetierte Zeile fürs Debuggen CRLF
        ''' CRLF
        # loop_thread = Thread(target=self.start_loop) CRLF
        # loop_thread.daemon = True  # Damit der Loop-Thread automatisch beendet
          wird, wenn die Hauptanwendung beendet wird CRLF
        # loop_thread.start() CRLF
        # webbrowser.get().open("http://127.0.0.1:8081") CRLF
        self.app.run(debug=True, host='0.0.0.0' ,port=8082) CRLF
        # serve(self.app, port=8082, threads=30) CRLF
        # self.cache = Cache(self.app, config={'CACHE_TYPE': 'simple',
          'CACHE_DEFAULT_TIMEOUT': 30}) CRLF
CRLF
CRLF
    ## @brief Setzt den aktuellen Status für das Webinterface. CRLF
    # @param status Neuer Statusstring. CRLF
    def getstatus(self, status): CRLF
        self.status = status CRLF
CRLF
CRLF
## @brief Hauptfunktion zum Starten der Weboberfläche. CRLF
# @details CRLF
#     Wird ausgeführt, wenn das Script direkt gestartet wird. CRLF
def main(): CRLF
    cb = CanbridgeWeb() CRLF
    cb.run() CRLF
CRLF
CRLF
## @brief Eintrittspunkt des Programms (wenn als Skript ausgeführt). CRLF
if __name__ == '__main__': CRLF
    main() CRLF
```